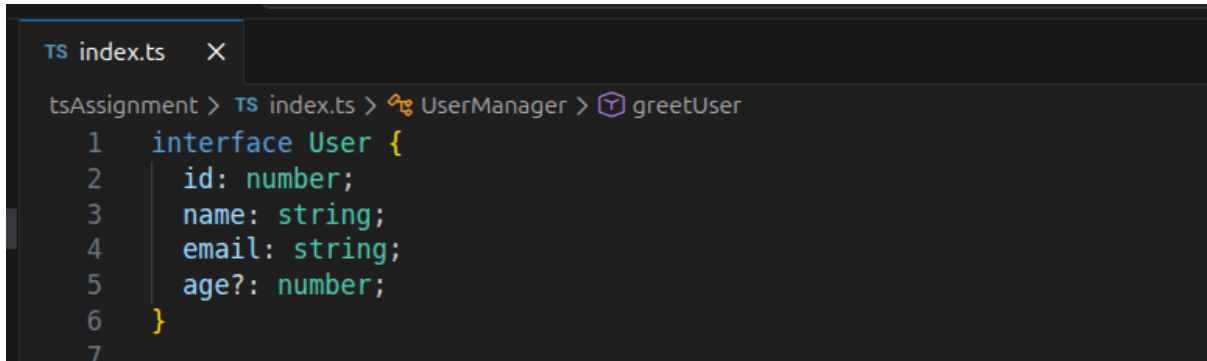


TS Assignment

Q1 Define an interface User with the following properties:

- id(number), name(string), email(string), age(number optional)



```
TS index.ts X
tsAssignment > TS index.ts > UserManager > greetUser
1  interface User {
2      id: number;
3      name: string;
4      email: string;
5      age?: number;
6  }
7
```

Q2 Create a class UserManager with:

- A private array users: User[] to store user data.
- A method addUser(user: User): void that adds a new user.
- A method removeUser(id: number): void that removes a user by ID.
- A method getUser(id: number): User | undefined that retrieves a user by ID.
- A method getAllUsers(): User[] that returns all users.

Code -

```
interface User {
  id: number;
  name: string;
  email: string;
  age?: number;
}

class UserManager {
  private users: User[] = [];

  addUser(user: User): void {
    this.users.push(user);
  }

  removeUser(id: number): void {
    this.users = this.users.filter((user) => user.id !== id);
  }

  getUser(id: number): User | undefined {
    for (let user of this.users) {
```

```
        if (user.id === id) {
            return user;
        }
    }
}

getAllUsers(): User[] {
    return this.users;
}

greetUser(name: string = "Guest"): string {
    const user = this.users.find((u) => u.name === name);
    return user ? `Hello ${user.name}` : "Hello Guest";
}

printUserDetails(targetUser: User): void {
    for (let user of this.users) {
        if (user.id === targetUser.id) {
            const { id, name, email, age } = user;
            console.log(id, name, email, age);
            return;
        }
    }
    console.log("User not found");
}

const obj1 : User = {
    id: 5,
    name: "aman",
    email: "aman@gmail.com",
    age: 20,
};

const obj2 : User = {
    id: 10,
    name: "dhruv",
    email: "dhruv@gmail.com",
    age: 22,
};
```

```

const obj3 : User = {
  id: 15,
  name: "Harsh",
  email: "harsh@gmail.com",
  age: 25,
};

const person1 = new UserManager();
person1.addUser(obj1);
person1.addUser(obj2);
person1.addUser(obj3);
console.log(person1.getAllUsers());

console.log(person1.getUser(10));
person1.removeUser(10);

console.log(person1.getAllUsers());

console.log(person1.greetUser("fdf"));
console.log(person1.greetUser("aman"));

person1.printUserDetails(obj1)

```

```

[Running] node "/home/aman-mittal/Desktop/typescript/tsAssignment/index.js"
[
  { id: 5, name: 'aman', email: 'aman@gmail.com', age: 20 },
  { id: 10, name: 'dhruv', email: 'dhruv@gmail.com', age: 22 },
  { id: 15, name: 'Harsh', email: 'harsh@gmail.com', age: 25 }
]
{ id: 10, name: 'dhruv', email: 'dhruv@gmail.com', age: 22 }
[
  { id: 5, name: 'aman', email: 'aman@gmail.com', age: 20 },
  { id: 15, name: 'Harsh', email: 'harsh@gmail.com', age: 25 }
]

```

Q3 Use Arrow Functions & Default Parameters

- Add a method greetUser = (name: string = "Guest"): string that returns a greeting message.

```
30  
31     greetUser(name: string = "Guest"): string {  
32         const user = this.users.find((u) => u.name === name);  
33         return user ? `Hello ${user.name}` : "Hello Guest";  
34     }  
35
```

Q4 Use Destructuring & Spread Operator

- Create a function printUserDetails(user: User): void that logs user details using object Destructuring.

```
35  
36     printUserDetails(targetUser: User): void {  
37         for (let user of this.users) {  
38             if (user.id === targetUser.id) {  
39                 const { id, name, email, age } = user;  
40                 console.log(id, name, email, age);  
41                 return;  
42             }  
43         }  
44         console.log("User not found");  
45     }  
46
```